Suite du brief : de votre modèle à une application en ligne

Le contexte

Vous avez déjà créé un modèle qui **prédit la catégorie d'un film (horreur ou comédie) à partir de sa description**. Mais il vit dans un notebook : seul vous pouvez l'utiliser, sur votre ordinateur.

Dans cette suite, vous allez **l'améliorer, le contrôler automatiquement, puis le mettre en ligne** : n'importe qui pourra envoyer le résumé d'un film à une adresse internet et recevoir la réponse.

Le travail se fait en **deux parties** :

- **Partie 1** : un notebook en 8 parties, qui produit un modèle et tout ce qu'il faut pour le réutiliser.
- **Partie 2** : à partir de ce modèle, une application, des tests, une boîte (conteneur), un robot GitHub et un déploiement sur Azure.

C'est ambitieux, et plusieurs notions seront nouvelles : c'est normal. Suivez les étapes **dans l'ordre**, et ne passez à la suivante qu'une fois le résultat attendu obtenu. Gardez une **preuve** (capture d'écran ou texte copié) de chaque résultat : vous les rendrez à la fin. Bloqué plus de 20 minutes ? Demandez de l'aide.

.

Partie 1 : le notebook (8 parties)

Vous repartez du **même jeu de données** (les résumés de films) et vous écrivez **un seul notebook**, organisé en 8 parties, avec un titre pour chacune. Aucun code n'est fourni : les mots à chercher sont dans la boîte à outils (fin du document).

1. Audit et nettoyage

Chargez le fichier. Comptez les lignes et les films de chaque catégorie. Regardez la longueur des textes. **Supprimez les espaces en trop et les doublons** (deux lignes identiques). Notez en une phrase ce que vous avez trouvé et ce que vous avez corrigé.
Pour comprendre : un même texte présent dans l'apprentissage et dans le test fausserait votre score, car le modèle aurait déjà vu la réponse.

2. Découpage du jeu de données

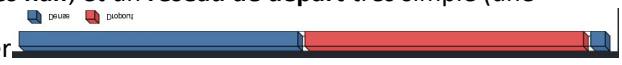
Séparez les textes en trois parties : **60 % apprentissage**, **20 % test**, **20 % production** (à garder de côté, vous n'y touchez pas). Gardez la même proportion d'horreur et de comédie dans chaque partie.
Pour comprendre : le test ne sert qu'une seule fois, à la toute fin. Tout ce que le modèle apprend, il l'apprend sur l'apprentissage seulement.

3. Représentation du texte (TF-IDF)

Un réseau ne lit pas des mots, il lit des nombres. Transformez chaque texte en nombres avec un **TF-IDF**. Apprenez le vocabulaire **sur l'apprentissage seulement**, puis transformez les trois parties avec ce même objet.
Pour comprendre : si le vocabulaire est appris avec le test, des informations du test « fuient » dans l'apprentissage et le score est trop flatteur.

4. Les points de comparaison

Entraînez trois modèles : une **régression logistique**, un **Bayes naïf**, et un **réseau de départ** très simple (une couche cachée, un peu de dropout, aucun réglage particulier.



). Mesurez le **F1** de chacun sur **5 graines différentes** (une graine = un tirage au hasard différent). Pour juger, utilisez une partie de l'apprentissage ; le test reste intact. Calculez la **marge de bruit** (voir la boîte à outils).
Pour comprendre : si deux scores diffèrent de moins que la marge de bruit, vous ne pouvez pas dire que l'un est meilleur que l'autre.

5. Recherche des réglages avec KerasTuner

Écrivez d'abord **une fonction qui construit un réseau** à partir de cinq réglages : nombre de couches, nombre de neurones, dropout, régularisation, taux d'apprentissage. Lancez ensuite **KerasTuner** (recherche au hasard, 20 essais).

6. Recherche des réglages avec Optuna

Refaites la même recherche avec **Optuna**, sur **exactement les mêmes réglages, les mêmes valeurs possibles, le même nombre d'essais (20)**, en réutilisant votre fonction. Comparez le meilleur score des deux outils, en tenant compte de la marge de bruit.
Pour comprendre : pour comparer deux outils, il faut les placer dans les mêmes conditions.

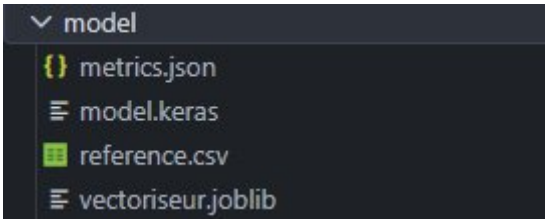
7. Verdict sur le test

Prenez les meilleurs réglages, réentraînez le réseau sur **tout l'apprentissage**, sur **5 graines**. Comparez-le à votre réseau de départ et au meilleur modèle simple. Puis mesurez-le sur le **test, une seule fois**. Concluez honnêtement : le réseau est-il meilleur, ou l'écart est-il plus petit que la marge de bruit ?

8. Sauvegarde

Créez un dossier (par exemple `model/`) et rangez-y :

- le **réseau** au format `.keras` ;
- le **TF-IDF** avec `joblib` ;
- un fichier **`metrics.json`** : version du modèle, F1 obtenu sur le test, taille du test, versions des bibliothèques (écrivez-le en **UTF-8**) ;
- un petit **échantillon de données de référence**.



Vous devez obtenir : un notebook qui s'exécute de haut en bas sans erreur, et un dossier `model/` complet. Mettez le notebook et le dossier dans votre dépôt GitHub.

.

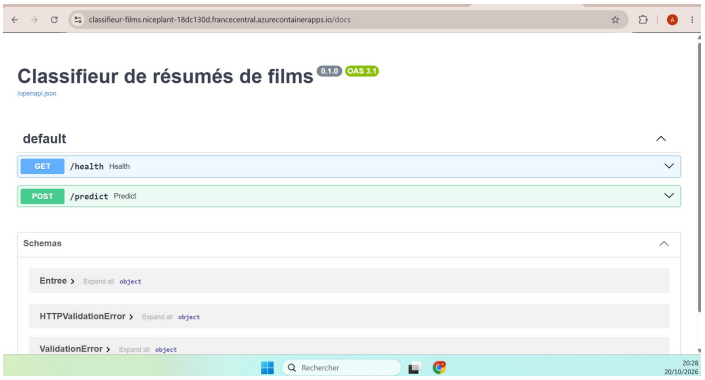
Partie 2 : de l'enregistrement à la mise en ligne

Une fois le modèle et tout le traitement des données enregistrés, vous allez devoir, dans l'ordre :

A. L'application

1. Écrivez une API (avec FastAPI) qui charge le modèle **une seule fois au démarrage**, avec deux routes :

- `/health` : répond « ok » et donne la version du modèle ;
- `/predict` : reçoit un texte, renvoie la classe, la probabilité et la version.



Si le texte est invalide (vide, trop court…), elle répond par un **message d'erreur clair**.
Pour comprendre : une API est un programme qui attend des questions envoyées par internet et y répond. Une route est une adresse précise de ce programme.

2. Lancez-la sur votre ordinateur et testez trois cas : un texte valide, un texte invalide, un cas limite (très court). FastAPI génère une page de test : cherchez comment l'ouvrir.
Vous devez obtenir : une classe pour le cas valide, un message d'erreur clair pour les deux autres.

3. Écrivez des tests automatiques : l'API répond, un cas valide marche, un cas invalide est refusé, le modèle atteint un score minimum. Lancez-les.
Pour comprendre : un test automatique est une vérification que l'ordinateur refait seul, à chaque modification.
Vous devez obtenir : tous les tests réussis.

4. Écrivez la barrière qualité. Mettez dans un fichier de configuration (par exemple `config.yaml`) un **F1 minimum** et une **taille de test minimale**. Écrivez un script qui **échoue** si le fichier `metrics.json` ne respecte pas ces seuils. Choisissez le seuil en tenant compte de la marge de bruit.
Vous devez obtenir : votre modèle est accepté, et un faux modèle (avec un F1 très bas que vous inventez) est refusé. Justifiez votre seuil en une phrase.

B. La boîte (conteneur)

5. Écrivez un `Dockerfile` (la recette qui décrit comment fabriquer la boîte) et un fichier **`.dockerignore`** (ce qu'il ne faut pas y mettre : données, tests, notebooks). Construisez l'**image**.
Pour comprendre : l'image est le « moule », le conteneur est la boîte lancée à partir du moule. Il doit être léger : notez sa taille.
Vous devez obtenir : une image construite.

6. Lancez un conteneur à partir de l'image, sur votre ordinateur, et refaites les trois tests de l'étape 2.
Vous devez obtenir : les mêmes réponses qu'à l'étape 2.

C. Le robot GitHub

7. Écrivez un workflow GitHub Actions avec deux jobs : **test** (installer le projet, lancer la barrière qualité et les tests) puis **image** (construire l'image et vérifier que l'API démarre dedans). Le second ne doit démarrer **que si** le premier a réussi. Envoyez tout avec `git push`.
*Pour comprendre : un workflow est un fichier qui donne des consignes au robot. Il doit être dans un dossier précis du dépôt : `.github/workflows` (avec un **s**).*
Vous devez obtenir : deux jobs avec une coche verte, dans l'onglet *Actions*.

8. Vérifiez que le robot sert à quelque chose : remontez volontairement le seuil de qualité **au-dessus** de ce que votre modèle peut atteindre, puis faites un `git push`. Remettez ensuite le bon seuil.
Vous devez obtenir : le job *test* en **rouge**, le job *image* qui **ne démarre pas**, puis tout repasse au **vert** après la correction.

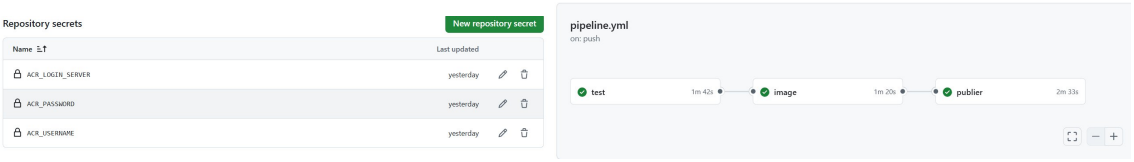
D. Azure

Vos droits. Sur le compte de formation, vous êtes **Contributeur** : vous pouvez *créer* des ressources, mais pas *donner des droits* à d'autres comptes. Conséquences : le robot GitHub pourra fabriquer l'image et la ranger dans le registre, mais pas ordonner à Azure de la mettre en ligne. **C'est donc vous qui déployez, à la main**, en indiquant l'identifiant du commit à mettre en ligne. Pour accéder au registre, utilisez son **utilisateur administrateur** (identifiant et mot de passe), et non l'option « identité managée », qui échouerait faute de droits.

9. Dans le portail Azure, créez un registre de conteneurs (offre *Basic*) dans votre groupe de ressources. Dans ses paramètres, activez l'**utilisateur administrateur**.
Pour comprendre : le registre est un entrepôt où l'on range des images.
Vous devez obtenir : l'adresse du registre, un nom d'utilisateur et un mot de passe.



10. Enregistrez ces trois valeurs comme secrets dans les paramètres de votre dépôt GitHub. Ajoutez au workflow un troisième job, **publier**, qui s'exécute après *image*, se connecte au registre avec les secrets et y envoie l'image, **étiquetée avec l'identifiant du commit**. Envoyez avec `git push`.
Pour comprendre : un secret est une valeur que GitHub garde cachée. Un mot de passe ne s'écrit jamais dans le code ni dans une conversation.
Vous devez obtenir : trois jobs verts, et dans le registre une image portant l'identifiant du commit.



11. Créez une application de conteneur à partir de cette image : authentification par **secret**, **1 processeur et 2 Go** de mémoire, entrée (*ingress*) **activée** et accessible depuis internet, **port** de votre API.
Pour comprendre : l'image doit exister dans le registre avant de créer l'application. Une mémoire trop faible fait redémarrer le conteneur sans fin (TensorFlow en consomme beaucoup).
Vous devez obtenir : une adresse publique.

12. Depuis votre navigateur, testez `/health` puis `/predict` sur l'adresse publique, avec les trois cas.
Vous devez obtenir : les mêmes réponses qu'en local. Montrez-le au formateur avant de continuer.

E. Faire vivre votre version

13. Ajoutez une petite route à votre API (par exemple `/hello`), faites un `git push`, attendez les trois jobs verts, puis **déployez la nouvelle image à la main** en indiquant l'identifiant de son commit.
Vous devez obtenir : `/hello` visible sur l'adresse publique, et le nouvel identifiant dans `/health`.



14. Revenez en arrière : redéployez l'**ancien** commit.
Vous devez obtenir : `/hello` a disparu et l'ancien identifiant est de retour dans `/health`.

F. Le ménage

15. Relevez le coût estimé de ce que vous avez créé, puis **supprimez vos ressources** : l'application, son environnement et le registre. Ne supprimez jamais un groupe de ressources partagé.
Pour comprendre : un registre est facturé tant qu'il existe, même si personne ne l'utilise.
Vous devez obtenir : une preuve (capture) que les ressources n'existent plus.

.

Boîte à outils

Des **mots à chercher** dans la documentation. Aucun exemple n'est donné : c'est volontaire.

Besoin	Mots à chercher
Préparer les données	`drop_duplicates`, `str.strip`, `train_test_split` (option `stratify`), `TfidfVectorizer`
Modèles de départ	`LogisticRegression`, `MultinomialNB`, `keras.Sequential`, `Dense`, `Dropout`, `EarlyStopping`, `f1_score`
Marge de bruit	`1,96 × √(p(1−p)/n)`, avec p le score et n la taille du test
Chercher les réglages	`keras_tuner.RandomSearch`, `optuna.create_study`, `trial.suggest_int`, `trial.suggest_float`
Sauvegarder, charger	`model.save`, `keras.models.load_model`, `joblib.dump`, `json.dump` (option `encoding`)
Écrire l'API	`FastAPI`, `uvicorn`, `pydantic` (pour vérifier les entrées)
Tests	`pytest`, `TestClient`, fichier `pytest.ini` (option `pythonpath`)
Image Docker	`Dockerfile`, `FROM`, `COPY`, `RUN`, `CMD`, `EXPOSE`, `.dockerignore`, `docker build`, `docker run`
Alléger l'image	image de base `slim`, `tensorflow-cpu`, versions des bibliothèques fixées
Robot GitHub	`.github/workflows`, `on: push`, `jobs`, `needs`, `runs-on`, `actions/checkout`, `actions/setup-python`, `docker/login-action`, `secrets`
Azure	`az login`, `az acr`, `az containerapp update`, `az containerapp logs`, entrée (*ingress*) externe, port cible
Identifier une version	`git rev-parse HEAD`, `github.sha`, étiquette d'image

Si ça ne marche pas

Tout le monde rencontre des erreurs. **La dernière ligne d'une erreur dit souvent ce qui ne va pas.**

Ce que vous voyez	Ce que ça veut dire	Où chercher
`No module named 'fastapi'` ou `'src'`	Python ne trouve pas une bibliothèque ou votre dossier	Le bon environnement ? Pour `src`, le fichier `pytest.ini`
Erreur de décodage UTF-8 en lisant un JSON	Fichier écrit sans préciser l'encodage (fréquent sous Windows)	L'encodage à l'écriture
« Accès refusé » en installant une bibliothèque	Un notebook ouvert bloque les fichiers	Fermez les notebooks et les noyaux
KerasTuner s'arrête (errno 22) sous Windows	Il écrit des fichiers de suivi que Windows refuse	Comment l'empêcher d'écrire ces fichiers
KerasTuner réclame `tensorboard`	Une bibliothèque manque	Son installation
Un score très bon mais suspect	Doublons entre apprentissage et test	La partie 1 du notebook
Le workflow n'apparaît pas dans *Actions*	Mauvais nom de dossier (`workflows` prend un **s**) ou Actions désactivées	L'orthographe exacte, les réglages du dépôt
`az login` échoue sous Windows	La fenêtre de connexion de Windows pose problème	L'option « code d'appareil » de `az login`
`$(...)` ne marche pas dans une commande	Syntaxe d'un autre terminal	Exécutez la commande en deux temps
`MANIFEST_UNKNOWN` en déployant	L'image n'est pas dans le registre, ou	Les images du registre (`az acr`

Ce que vous voyez	Ce que ça veut dire	Où chercher
	mauvaise étiquette	`repository list`)
`roleAssignments/write` refusé	Votre rôle ne permet pas de donner des droits	Utilisez l'utilisateur administrateur du registre
`Not Found` sur l'adresse publique	L'API marche, mais cette adresse n'est pas une route	Ajoutez `/health` ou `/docs`
Pas d'adresse publique	L'entrée (*ingress*) est désactivée	Les paramètres réseau de l'application
Le conteneur redémarre sans cesse	Mémoire insuffisante	La mémoire allouée
Nom de registre refusé	Il doit être unique au monde, en minuscules, sans tiret	Un autre nom

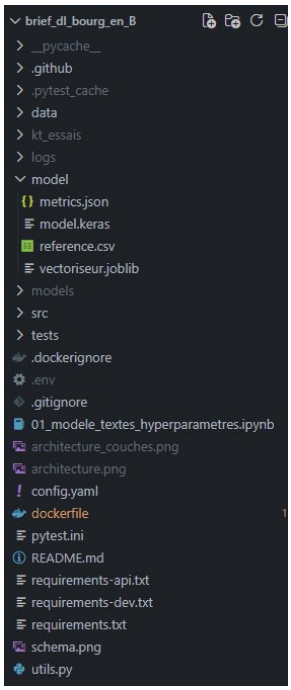
Ce que vous rendez

1. Un **dépôt GitHub** contenant : le notebook, le dossier `model/`, l'API, les tests, le fichier de seuils et le script de barrière qualité, les fichiers de dépendances, le `Dockerfile`, le `.dockerignore`, le workflow, et un `README.md` qui explique comment tout lancer.
2. Vos **preuves** : tests en local, conteneur en local, chaîne verte, **chaîne rouge avec le seuil impossible**, adresse publique, `/health` avec le commit, ajout puis retrait de la route `/hello`, suppression des ressources.
3. Une **fiche d'une page**, avec vos mots : le trajet de votre modèle du notebook jusqu'à internet ; pourquoi ce seuil de qualité ; pourquoi vous déployez à la main ; ce qu'il faudrait pour que le déploiement soit automatique ; les limites de votre travail.

Lexique

Mot	Sens
Hyperparamètre	Réglage choisi avant l'entraînement (nombre de neurones, dropout…). Le modèle ne l'apprend pas seul.
Graine	Nombre qui fixe le hasard. Plusieurs graines = plusieurs tirages, pour voir si le score est stable.
F1	Score entre 0 et 1 qui résume la qualité d'une classification. 1 = parfait.
TF-IDF	Transformation d'un texte en nombres, selon les mots utilisés et leur rareté.
API	Programme qui répond à des questions envoyées par internet.
Test automatique	Vérification que l'ordinateur refait seul à chaque modification.
Barrière qualité	Script qui refuse un modèle trop mauvais.
Image / conteneur	L'image est le moule, le conteneur est la boîte lancée à partir de l'image.
Dockerfile	La recette qui décrit comment fabriquer l'image.
Commit	Une version enregistrée de votre code, avec un identifiant unique.
Workflow, job	Le workflow est le fichier de consignes du robot GitHub ; un job est un bloc de tâches.
CI / CD	Vérifier et fabriquer automatiquement (CI) ; mettre automatiquement en ligne (CD).
Secret	Valeur sensible que GitHub garde cachée.
Registre	Entrepôt où l'on range des images.
Déployer	Mettre une version en ligne.
Retour arrière	Remettre en ligne une version précédente.

Exemple d'arborescence projet final :


brief_dl_bourg_en_B
__pycache__
.github
.pytest_cache
data
kt_essais
logs
model
metrics.json
model.keras
reference.csv
vectoriseur.joblib
models
src
tests
.dockerignore
.env
.gitignore
01_modele_textes_hyperparametres.ipynb
architecture_couches.png
architecture.png
config.yaml
dockerfile
pytest.ini
README.md
requirements-api.txt
requirements-dev.txt
requirements.txt
schema.png
utils.py